

**Lab Report No 5**  
**Implementation of Image Histogram and**  
**Equalization**  
**Digital Image Processing**



**Submitted By:**

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

**Submitted to:**

Engr. Hareem Khan

**Dated:**

25<sup>th</sup> Oct, 2024

**Department of Computer Engineering,**  
**HITEC University, Taxila**

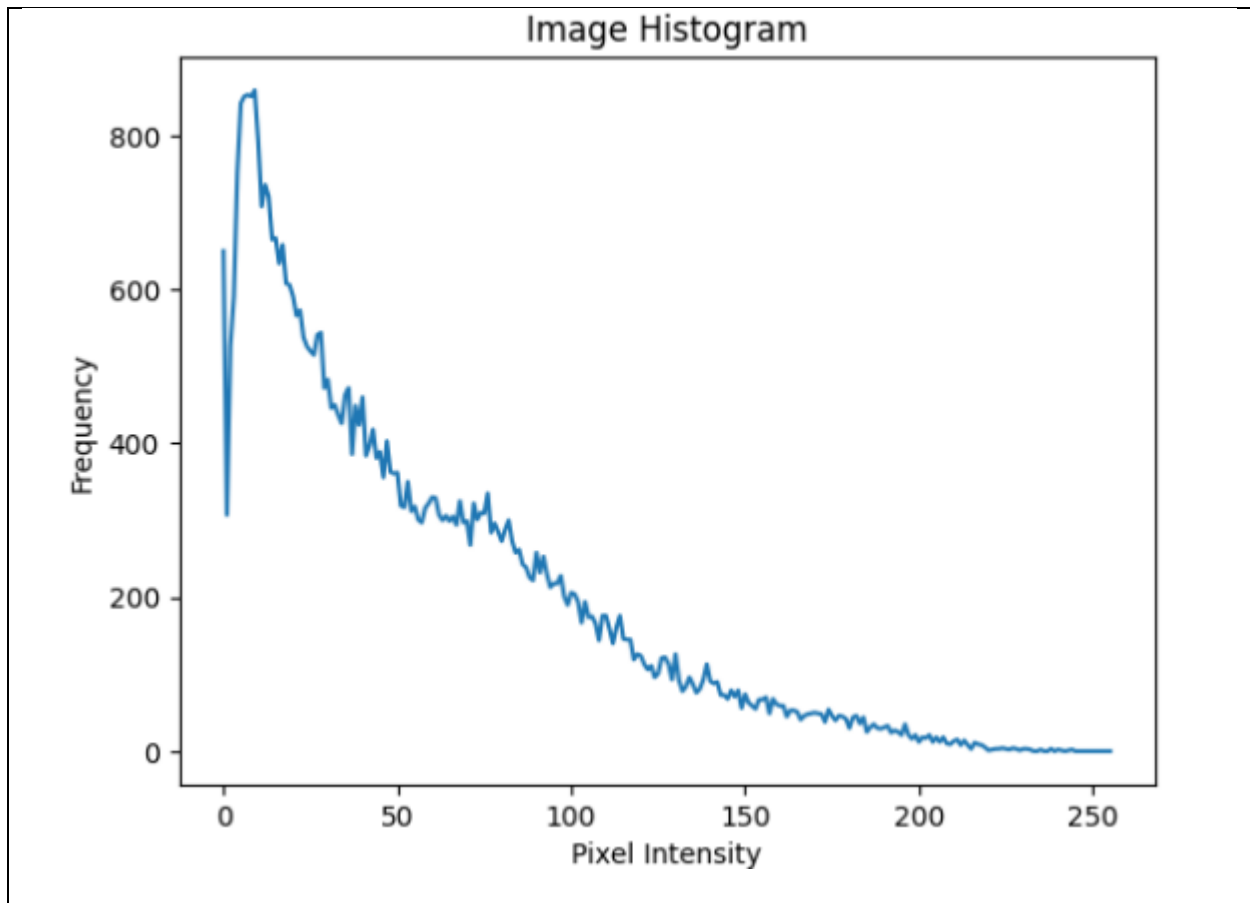
---

### Lab Example No 1:

---

#### Solution:

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Brief description (3-5 lines)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| An image histogram is a visual representation of the tonal distribution in a digital image, plotting the number of pixels for each tonal value, allowing viewers to quickly understand the entire tonal distribution in a specific image.                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>The code</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <pre>import cv2 import numpy as np import matplotlib.pyplot as plt # Load the image in grayscale image = cv2.imread('download.jpeg', 0) # Calculate the histogram using numpy.histogram() # 'bins=256' means we want 256 intensity levels (for grayscale images, 0 to 255) # 'range=(0, 256)' means we are considering pixel intensities in this range hist, bin_edges = np.histogram(image, bins=256, range=(0, 256)) #histogram = cv2.calcHist([image], [0], None, [256], [0, 256]) # Plot the histogram plt.plot(bin_edges[0:-1], hist) plt.title('Image Histogram') plt.xlabel('Pixel Intensity') plt.ylabel('Frequency') plt.show()</pre> |
| <b>The results (Screenshot)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |



---

### Lab Example No 2:

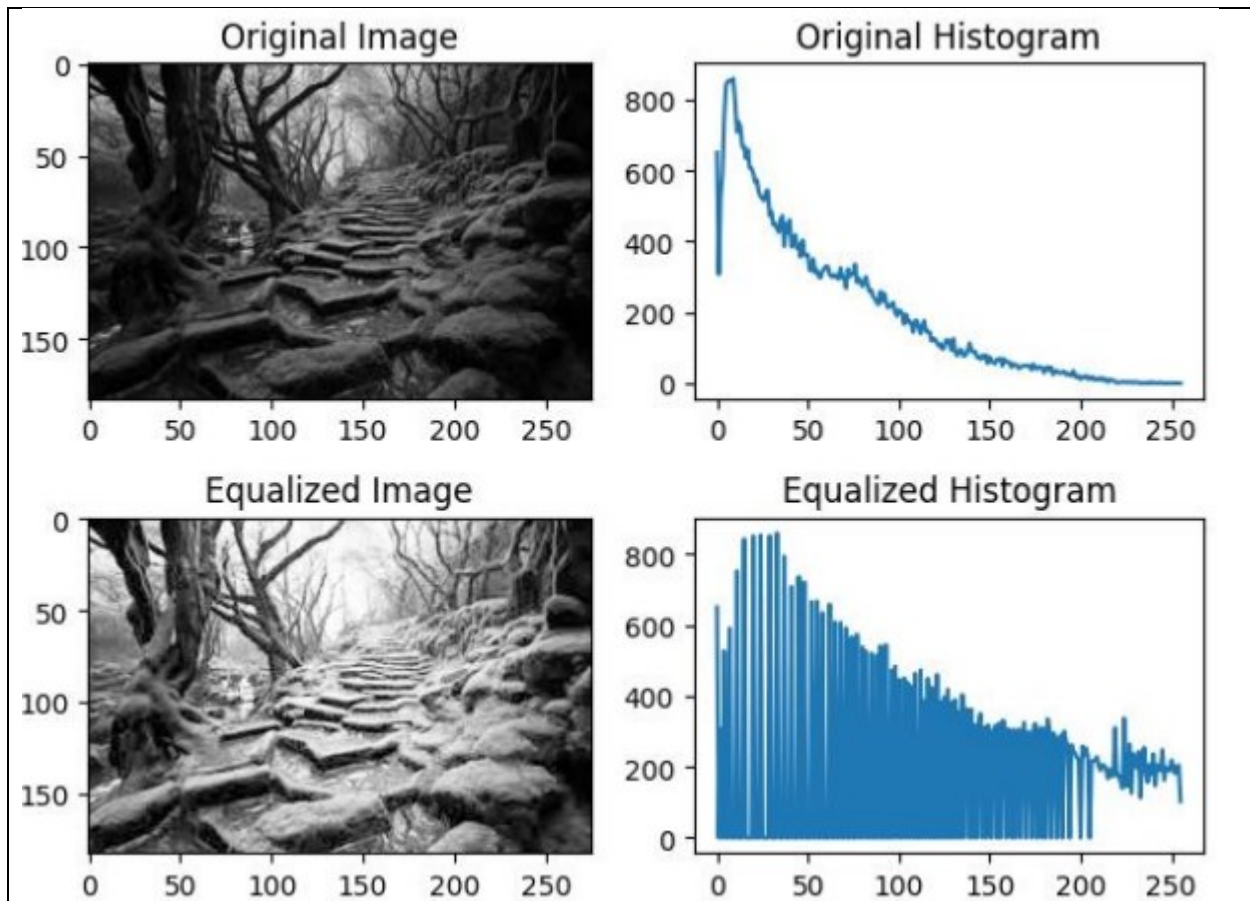
---

#### Solution:

| Brief description (3-5 lines)                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Histogram equalization:</b><br>Histogram of an image shows the frequency of different intensities values present in the image.<br>This gives a clear idea of what intensities dominate the image.<br>Histogram equalization is a technique that uses this information to enhance the contrast using the probability of a certain pixel to occur. The Image obtained will be Histogram Equalized Image with high contrast. |
| <b>The code</b>                                                                                                                                                                                                                                                                                                                                                                                                              |
| <pre>import cv2</pre>                                                                                                                                                                                                                                                                                                                                                                                                        |

```
import numpy as np
import matplotlib.pyplot as plt
# Load the image in grayscale
image = cv2.imread('download.jpeg', 0)
# Perform Histogram Equalization
equalized_image = cv2.equalizeHist(image)
# Calculate the histogram of the equalized image
equalized_histogram = cv2.calcHist([equalized_image], [0], None,
[256], [0,
256])
# Display the original and equalized image side by side
plt.subplot(2, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.subplot(2, 2, 2)
plt.plot('histogram')
plt.title('Original Histogram')
plt.subplot(2, 2, 3)
plt.imshow(equalized_image, cmap='gray')
plt.title('Equalized Image')
plt.subplot(2, 2, 4)
plt.plot(equalized_histogram)
plt.title('Equalized Histogram')
plt.show()
```

**The results (Screenshot)**




---

### Lab Task No 1:

---

#### Solution:

##### Brief description (3-5 lines)

Apply Histogram equalization on the given image by applying above steps.

- Calculate the histogram of the image save it as Figure\_1.jpg.
- Calculate probability density function (PDF) from the histogram and save it as Figure\_2.jpg.  $PDF = H/(R*C)$ . Where H is the Histogram and R and C is the number of Rows and Columns of the image respectively.
- Calculate cumulative density function (CDF) and save it as Figure\_3.jpg.
- Multiply the Cumulative PDF with 255 to find the transformation function then save it as Figure\_4.jpg
- From the transformation function, replace the gray levels of the image to create contrast enhanced (histogram equalized) image and save it as Figure\_5.jpg
- Calculate the histogram of the output image save it as Figure\_6.jpg.

**The code**

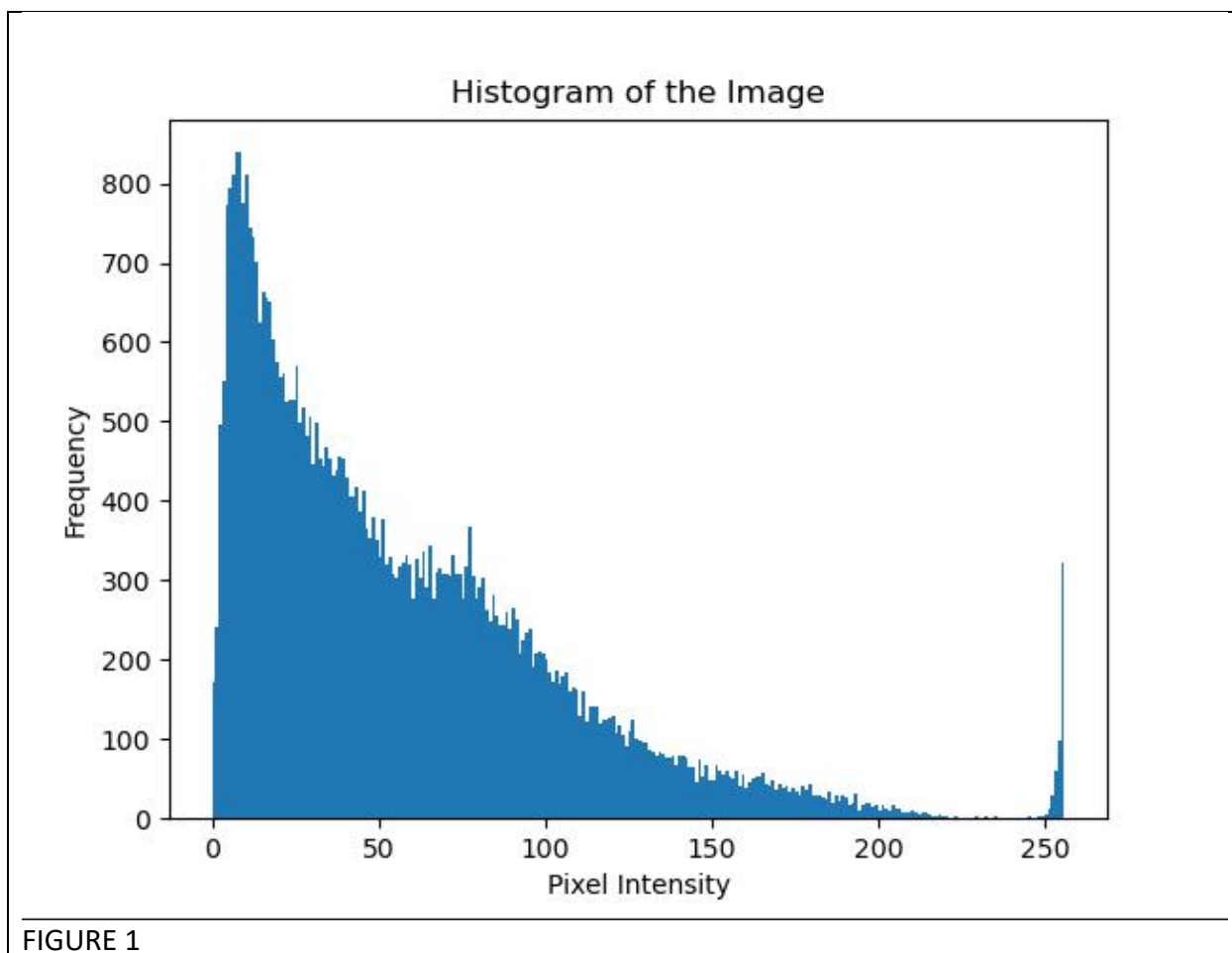
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image
image = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
# Calculate the histogram of the image
hist, bins = np.histogram(image.flatten(), 256, [0, 256])
# Save the histogram as Figure_1.jpg
plt.figure()
plt.hist(image.flatten(), 256, [0, 256])
plt.title("Histogram of the Image")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.savefig("Figure_1.jpg")
# Calculate the probability density function (PDF)
rows, cols = image.shape
pdf = hist / (rows * cols)
# Save the PDF as Figure_2.jpg
plt.figure()
plt.plot(pdf)
plt.title("Probability Density Function")
plt.xlabel("Pixel Intensity")
plt.ylabel("Probability")
plt.savefig("Figure_2.jpg")
# Calculate the cumulative density function (CDF)
cdf = np.cumsum(pdf)
# Save the CDF as Figure_3.jpg
plt.figure()
plt.plot(cdf)
plt.title("Cumulative Density Function")
plt.xlabel("Pixel Intensity")
plt.ylabel("Cumulative Probability")
plt.savefig("Figure_3.jpg")
```

```

# Multiply the Cumulative PDF with 255 to find the transformation
function
transform_function = cdf * 255
# Save the transformation function as Figure_4.jpg
plt.figure()
plt.plot(transform_function)
plt.title("Transformation Function")
plt.xlabel("Pixel Intensity")
plt.ylabel("Output Intensity")
plt.savefig("Figure_4.jpg")
# Apply the transformation function to the image
equalized_image = np.uint8(np.interp(image.flatten(), np.arange(256),
transform_function))
equalized_image = np.reshape(equalized_image, image.shape)
# Save the contrast enhanced image as Figure_5.jpg
cv2.imwrite("Figure_5.jpg", equalized_image)
# Calculate the histogram of the output image
hist, bins = np.histogram (equalized_image.flatten(), 256, [0, 256])
# Save the histogram of the output image as Figure_6.jpg
plt.figure()
plt.hist(equalized_image.flatten(), 256, [0, 256])
plt.title("Histogram of the Equalized Image")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.savefig("Figure_6.jpg")

```

**The results (Screenshot)**





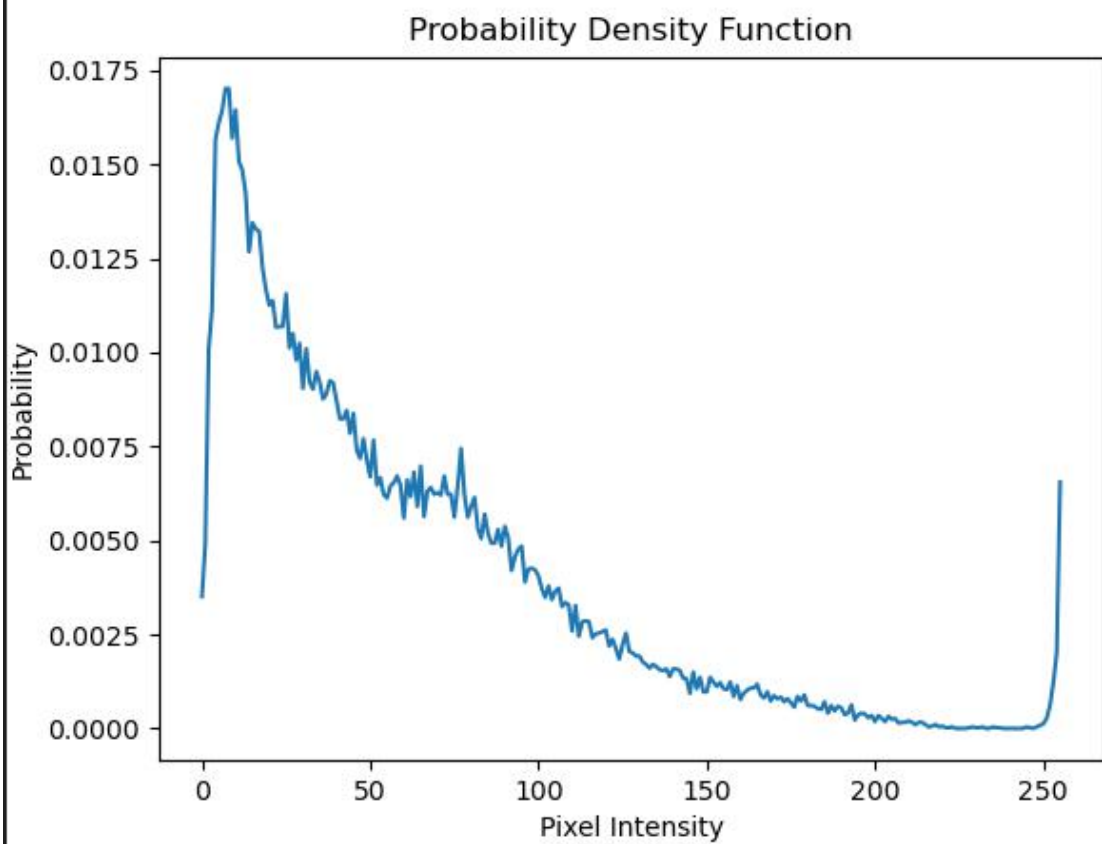
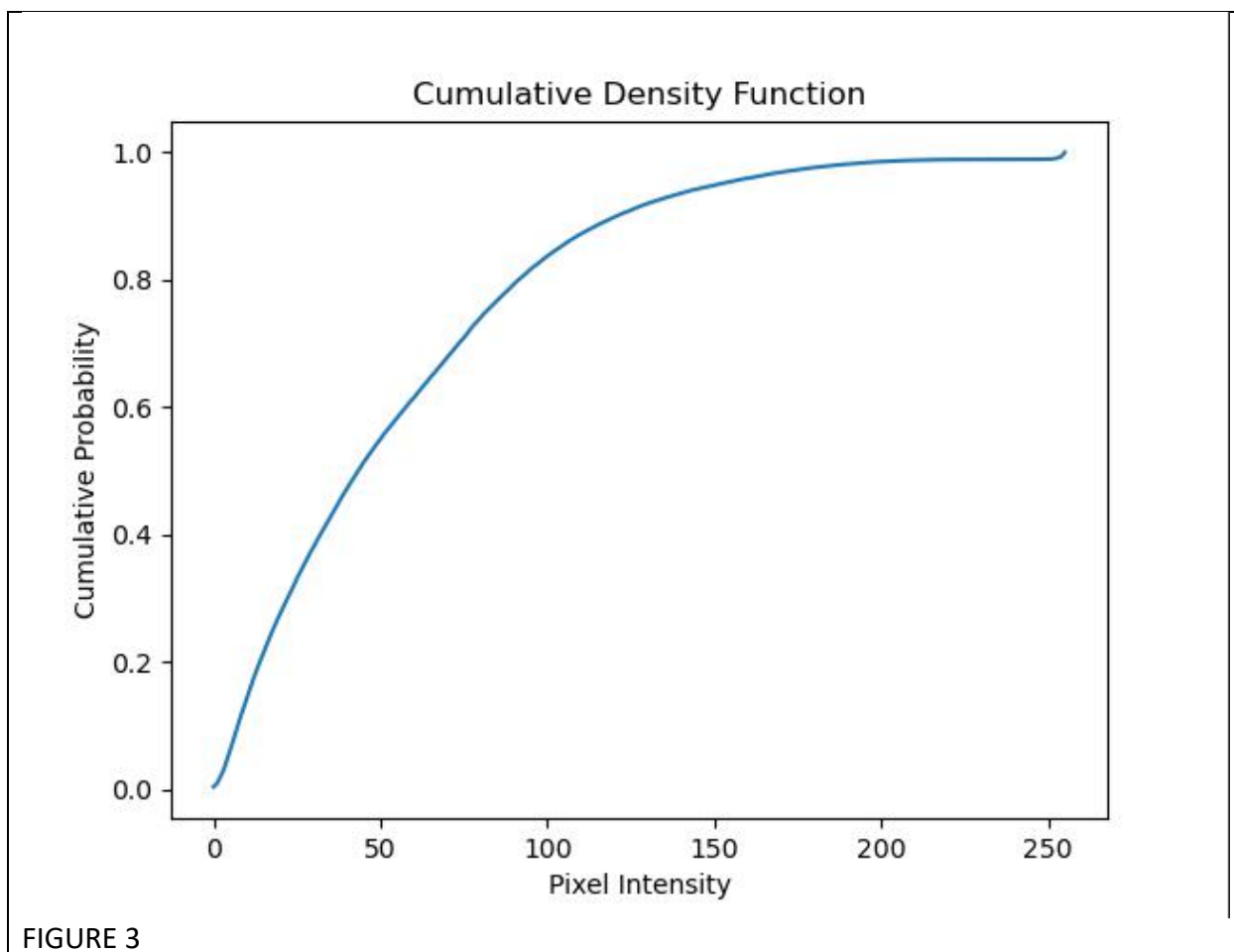


FIGURE 2



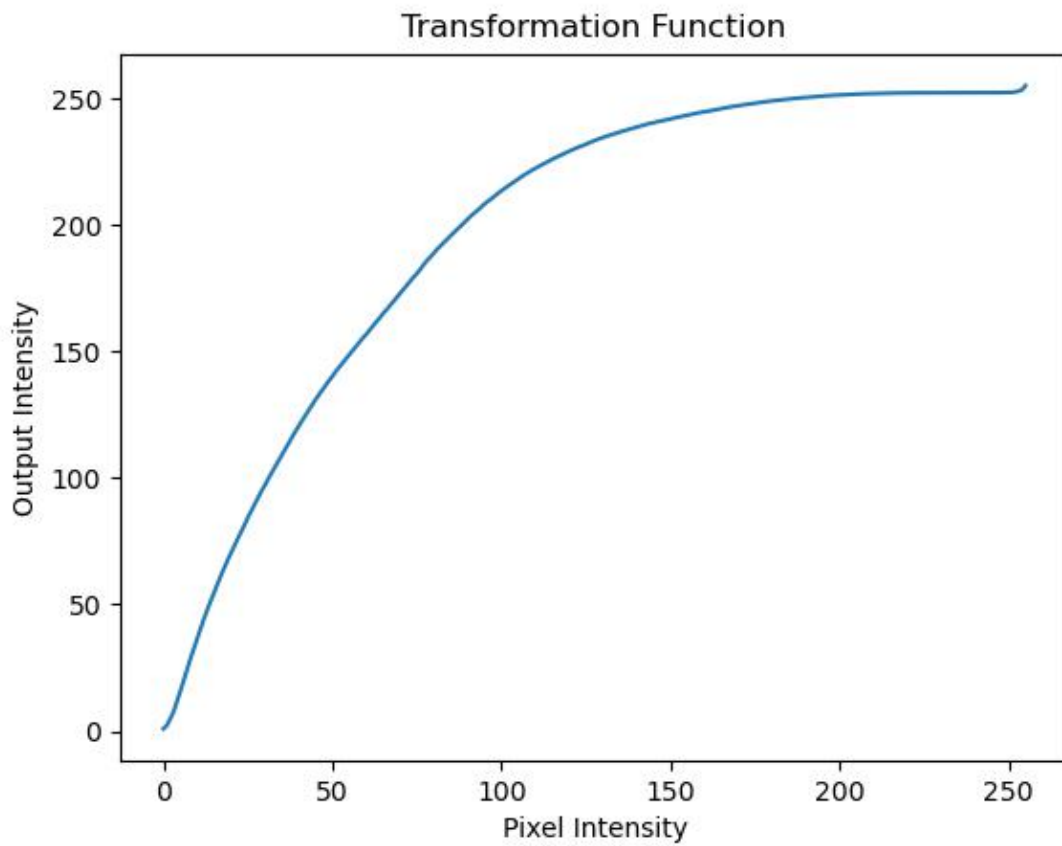
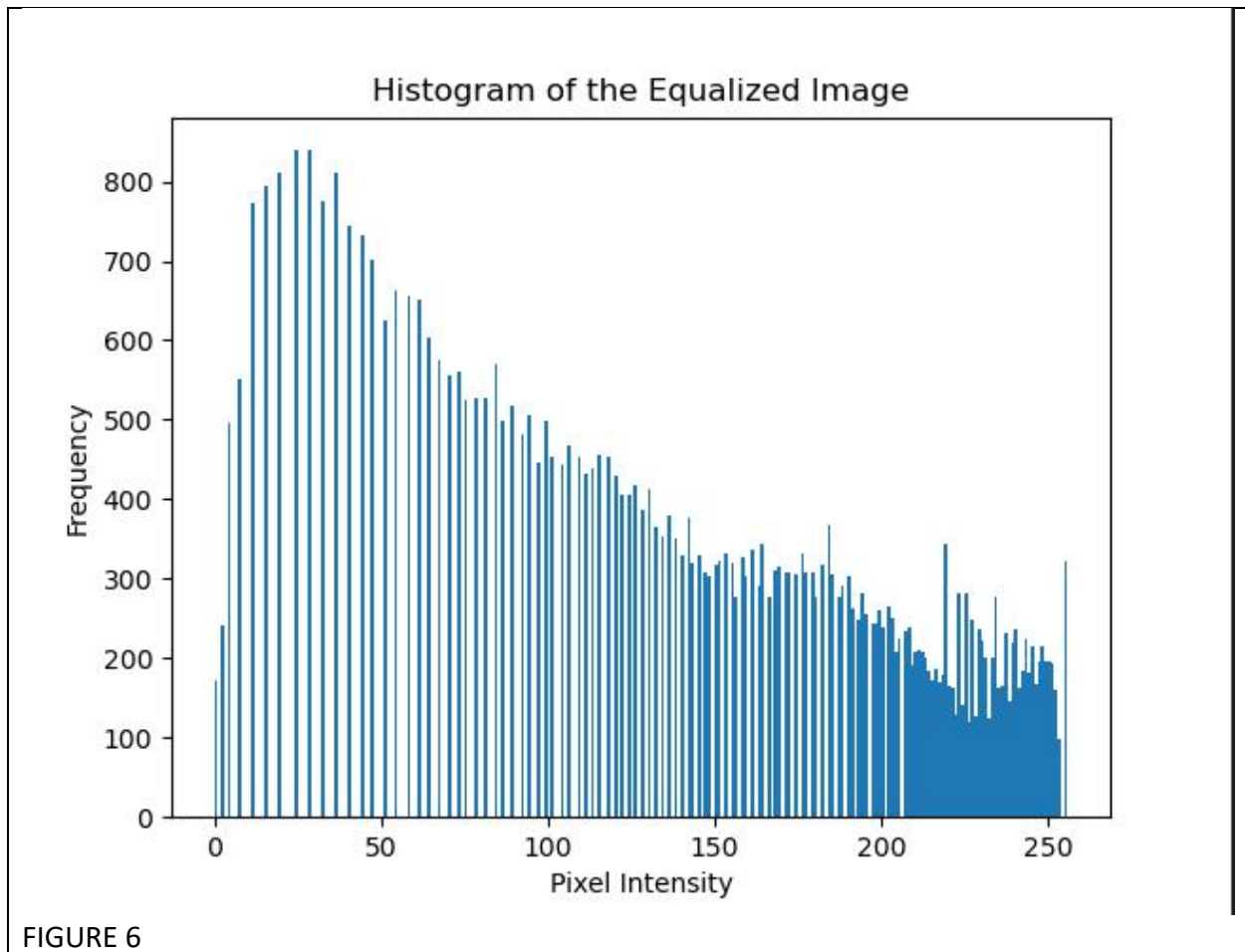


FIGURE 4



FIGURE 5



---

### Lab Task No 2:

---

#### Solution:

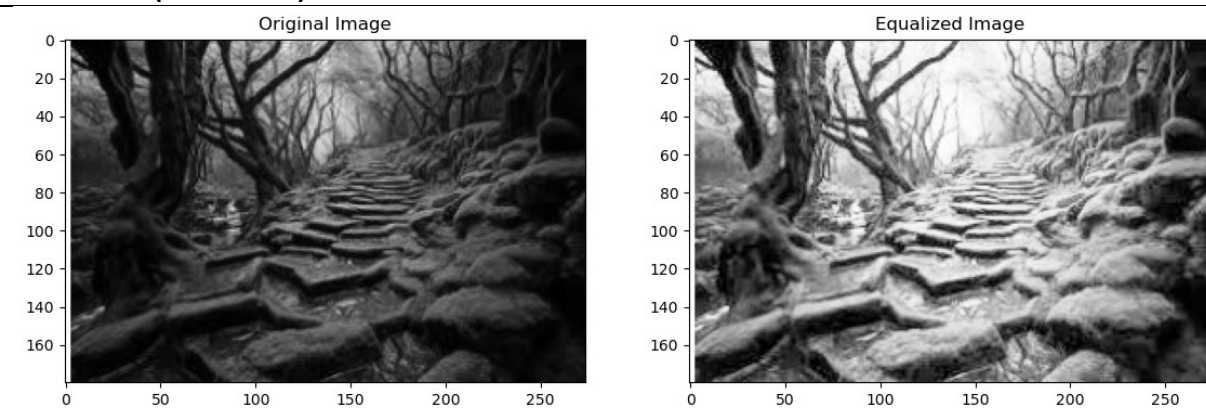
| Brief description (3-5 lines)                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------|
| For the given image                                                                                                          |
| i. Calculate histogram of the given image using build-in functions of numpy or OpenCV.                                       |
| ii. Calculate histogram equalization of the given image using build-in functions of numpy or OpenCV and display your output. |
| The code                                                                                                                     |
| <pre>import cv2 import numpy as np import matplotlib.pyplot as plt</pre>                                                     |

```

# Load the image
img = cv2.imread('Screenshot 2024-10-25 103016.jpg')
# Convert to grayscale
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Calculate the histogram
hist = cv2.calcHist([gray_img], [0], None, [256], [0, 256])
# Calculate histogram equalization
equalized_img = cv2.equalizeHist(gray_img)
# Display the original and equalized images
plt.subplot(1, 2, 1)
plt.imshow(gray_img, cmap='gray')
plt.title('Original Image')
plt.subplot(1, 2, 2)
plt.imshow(equalized_img, cmap='gray')
plt.title('Equalized Image')
plt.show()

```

#### The results (Screenshot)



#### Conclusion

In conclusion, Image histogram and equalization improve image quality by enhancing contrast and brightness levels. This technique, particularly beneficial in low-contrast images, enhances detail visibility. Histogram equalization is computationally efficient, making it suitable for real-time applications. Its effectiveness is demonstrated in medical imaging, remote sensing, and photography, demonstrating its potential in various imaging fields.